Java Asciidoctor Application Architecture

Table of Contents

1. Purpose and scope . . . 1
2. Decisions and open choices . . . 1
3. Java and dependency selection . . . 2
4. Application structure . . . 2
5. Initial conversion workflow . . . 3
6. Diagram integration . . . 3
7. HTML delivery . . . 4
8. PDF delivery . . . 4
9. Windows distribution . . . 4
10. Validation strategy . . . 5
11. Remaining architectural questions . . . 5
12. Reference documentation . . . 5

1. Purpose and scope

This document summarizes the architectural discussion for a standalone Java application running locally on Windows. Document generation is its first capability; additional functionality may be introduced later. The project remains in the research and architectural design phase. No application implementation or dependency installation has been authorized.

The initial capability converts one AsciiDoc entry document, referencing external PlantUML files, into HTML, PDF, or both. The application provides a Swing desktop GUI. The command-line boundary is reserved as an interface stub only; CLI functionality is deferred. Document preview is not required.

2. Decisions and open choices

Area	Direction	Status
Platform	Local, standalone Windows application	Agreed
Build	Maven	Agreed
Java	Java 21 LTS	Agreed; migrate later if necessary
Desktop GUI	Swing	Agreed

Area	Direction	Status
Command line	Interface stub only	Agreed; implementation and library selection deferred
AsciiDoc processing	AsciidoctorJ	Agreed
PDF generation	Asciidoctor PDF through AsciidoctorJ PDF integration	Recommended
Diagram integration	AsciidoctorJ Diagram	Recommended; dependency details to verify
Diagram rendering	PlantUML JAR and Graphviz	Accepted
HTML delivery	One self-contained HTML file with embedded images and styling	Agreed
Preview	None	Agreed
Distribution	Bundle a Java runtime and preferably Graphviz	Recommended; packaging details open

AsciiDoc is the source document format. Asciidoctor is the processor, and AsciidoctorJ exposes it through a Java API.

3. Java and dependency selection

Java 21 LTS is the selected baseline for compilation and execution. A migration to a newer Java release can be considered later if necessary. Bundling the runtime avoids depending on the Java version installed on a user's machine.

AsciidoctorJ uses JRuby internally, adding runtime size and initialization cost. The application should manage renderer initialization and lifecycle explicitly.

Exact library versions have not been selected. Before selecting them, verify compatibility among the JDK, AsciidoctorJ, PDF integration, Diagram integration, and PlantUML. Inspect transitive dependencies before adding a separate PlantUML dependency, to avoid duplicate or conflicting versions. Graphviz is an external executable rather than a Maven Java library.

4. Application structure

Use a small modular monolith with clear package boundaries:

- Application core: conversion requests, validation, input and output paths, options, and diagnostic results.
- Rendering integration: AsciidoctorJ initialization, PDF and diagram integration, themes, resource resolution, and Graphviz configuration.
- Desktop interface: Swing controls, file selection, progress, and diagnostics.

- Command-line boundary: an interface stub only, with implementation deferred. No CLI parser dependency is selected or required initially.

The GUI calls the conversion service; any future CLI implementation should use the same service. AsciidoctorJ-specific types should stay inside the rendering integration boundary. A future CLI should run without initializing Swing.

Start with packages rather than requiring separate Maven modules for every boundary. Introduce modules when stronger dependency enforcement or independent reuse justifies them. Future functionality does not yet justify a plugin framework, database, Spring Boot, or dependency-injection framework. Explicit construction should be sufficient initially.

CLI commands, options, help, exit codes, and functionality will be decided later.

5. Initial conversion workflow

1. Select or supply one `.adoc` entry document.
2. Choose HTML, PDF, or both.
3. Select an output directory.
4. Generate the requested documents.
5. Report success, warnings, and actionable errors.

The entry document references its PlantUML files; users should not need to select every diagram separately. Resolve relative resources from the document location rather than the process working directory, while respecting any explicitly configured resource paths. Keep generated intermediate images and temporary files out of the source directory.

Start with sequential conversion. Run conversion outside the Swing event-dispatch thread so the interface remains responsive. Parallel rendering, caching, and advanced cancellation behavior can be assessed when requirements justify them.

6. Diagram integration

Prefer Asciidoctor Diagram to connect diagram references in AsciiDoc with PlantUML rendering. Having the PlantUML JAR on the classpath alone does not provide that document integration.

An alternative is to render PlantUML images in an application-controlled preprocessing stage, then convert AsciiDoc referencing those images. This provides more explicit control but makes the application responsible for dependency tracking, image naming, caching, and diagnostic handling. Use the existing integration first unless concrete limitations justify custom orchestration.

PlantUML with Graphviz is acceptable for the initial architecture. Some diagram layouts require Graphviz; alternative engines such as Smetana were discussed but are not the selected baseline. Evaluate SVG for HTML diagrams because it preserves sharpness when zoomed. Validate PDF diagram rendering separately rather than assuming identical format support and behavior.

7. HTML delivery

Generate one self-contained HTML file by default. Asciidoctor supports embedding images through the `data-uri` attribute and embedding stylesheets.

Benefits include simple copying and sharing, no accompanying assets folder, and fewer broken image paths when moving the output.

Trade-offs include:

- Base64-encoded image payloads are approximately one third larger than their original binary data. This overhead applies to the encoded payload, not necessarily the entire document.
- Updating one diagram requires regenerating and redistributing the HTML file.
- Very large embedded documents can take longer to open and consume more memory.
- Images cannot be independently cached or shared across documents as external assets can.
- Extracting individual diagrams is less convenient.
- Some hosting environments may restrict data URIs through content policies.

Embedding images alone does not guarantee offline independence. Stylesheets, fonts, icons, scripts, and other resources must also be embedded, omitted, or otherwise made independent of network access. Opening the exported document with the network disconnected should be an acceptance criterion.

An HTML file with an assets folder remains a possible future export mode if actual output sizes or publishing requirements justify it. It is not required for the initial scope.

8. PDF delivery

Use Asciidoctor PDF through its JVM integration as the initial approach. PDF generation is a separate rendering pipeline, not automatically a printout of the HTML output. HTML CSS and PDF themes require separate styling decisions; identical appearance should not be assumed.

If close visual equivalence with browser output becomes a requirement, browser-based PDF generation is an alternative to reassess. No embedded browser or PDF viewer is needed for the current scope.

9. Windows distribution

Provide a GUI launcher initially. A console launcher is deferred with the CLI implementation. Bundle a Java runtime for predictable execution without requiring a separate Java installation. Prefer bundling Graphviz to requiring manual installation, subject to confirming redistribution requirements and packaging behavior.

The installer format, Java 21 runtime vendor and patch version, Graphviz version, and executable discovery strategy remain open. Windows-only support reduces packaging scope but does not

remove the need to validate runtime and external executable discovery on a clean machine.

10. Validation strategy

Use real conversion fixtures containing text, external PlantUML files, and representative resource paths. Prioritize these checks:

- HTML and PDF contain expected text and rendered diagrams.
- Conversion works when launched from a different working directory.
- Missing files, invalid diagrams, and unavailable Graphviz produce useful diagnostics.
- Self-contained HTML opens correctly without network access.
- The Swing interface remains responsive during conversion.
- When a CLI is implemented, verify that it uses the shared conversion service consistently with the GUI.
- A packaged application runs on Windows without a separately installed JDK.

Avoid byte-for-byte PDF comparisons because metadata and rendering details can make them brittle. Prefer content assertions and selected visual checks. Specific testing frameworks and Maven plugins have not yet been selected.

11. Remaining architectural questions

- Dependency versions compatible with Java 21.
- CLI implementation, library, and command structure, deferred to a later phase; only an interface stub is in the initial scope.
- PDF theme, HTML styling, fonts, and diagram formats.
- Output naming and overwrite behavior.
- Trust model for source documents, includes, and external resource access.
- Windows packaging format and bundling of Graphviz.
- Representative document sizes and acceptable conversion time.

12. Reference documentation

- Asciidoctor documentation overview
- AsciidoctorJ installation
- Asciidoctor Diagram
- Embedding images in HTML
- HTML stylesheet behavior
- PlantUML and Graphviz

- PlantUML layout engines